End-to-End MLOps

Heart Disease Risk Classifier

Machine Learning Operations (MLOps) AIMLCZG523

Assignment 01 | UCI Heart Disease Cleveland Dataset

A reproducible machine-learning solution covering data acquisition, exploration, modelling, experiment tracking, CI/CD, containerisation, deployment, serving, and monitoring.

Prepared for academic submission

1. Executive Summary

This project implements a binary classifier that predicts whether a patient record indicates the presence of heart disease. The solution is deliberately structured as a production-style MLOps project: data acquisition is reproducible, preprocessing is part of the serialized model pipeline, experiments are tracked with MLflow, quality checks run in GitHub Actions, and inference is exposed as a container-ready FastAPI service.

Two classification candidates are evaluated using stratified five-fold cross-validation. Logistic Regression is selected by mean ROC-AUC, then evaluated on a stratified holdout set. The validated local run selected Logistic Regression with cross-validated ROC-AUC 0.9154 and holdout ROC-AUC 0.9578.

Important limitation: this is an educational model and not a medical diagnostic system. Its confidence value is not a calibrated clinical risk score.

2. Data Acquisition and Exploratory Analysis

scripts/download_data.py downloads the Cleveland subset directly from the UCI Machine Learning Repository. The original question-mark values are converted to missing values. The original num field is mapped to a binary target: zero is the negative class and values one through four are the positive class.

The resulting dataset contains 303 rows and 13 predictors: age, sex, chest-pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, oldpeak, slope, number of major vessels, and thalassemia status.

scripts/eda.py generates feature histograms, a class-balance chart, and a correlation heatmap under artifacts/eda/. These plots are generated from the downloaded dataset and can be included as screenshots in the final submission.

The EDA stage is intentionally separate from model fitting. It is used for understanding distributions, missingness, class balance, and relationships; it does not fit transformations using the holdout test data.

3. Feature Engineering and Reproducibility

The reusable preprocessing transformer is defined in src/pipeline.py. Numeric variables are median-imputed and standardised with StandardScaler. Categorical variables are imputed with the most frequent value and encoded with OneHotEncoder(handle_unknown='ignore').

The transformer is placed inside a scikit-learn Pipeline with the estimator. This prevents data leakage because each cross-validation fold fits its preprocessing only on the corresponding training partition. The complete fitted pipeline is saved as artifacts/model.joblib.

requirements.txt pins the main runtime packages. The project can be recreated from a clean Python 3.10 or 3.12 environment using the commands in docs/SETUP.md. The API validates input ranges with Pydantic before the model receives a request.

4. Model Development and Evaluation

The project compares Logistic Regression and Random Forest. Logistic Regression uses max_iter=1000 and balanced class weights. Random Forest uses 300 estimators, maximum depth 6, balanced class weights, and a fixed random seed.

Model	CV Accuracy	CV Precision	CV Recall	CV ROC-AUC
Logistic Regression	0.8449	0.8565	0.7984	0.9154
Random Forest	0.8251	0.8377	0.7696	0.9123

The selection criterion is mean cross-validated ROC-AUC because it evaluates ranking quality across classification thresholds. Accuracy, precision, and recall are also reported because the cost of missed positive cases is relevant to this problem.

The selected Logistic Regression model achieved holdout accuracy 0.8689, precision 0.8125, recall 0.9286, and ROC-AUC 0.9578 on the fixed stratified 20% split.

5. Experiment Tracking and Model Packaging

scripts/train.py creates separate MLflow runs for the two candidate models. Each candidate run logs the model name, number of folds, cross-validation accuracy, precision, recall, and ROC-AUC. The fitted candidate pipeline and serialized artifact are also logged.

A final holdout run records the selected model, holdout metrics, model.joblib, confusion_matrix.csv, and roc_curve.png. This makes the model choice auditable and allows an evaluator to compare the candidates through the MLflow UI.

The packaged artifact includes both feature transformations and the classifier. Inference therefore does not depend on a separately maintained preprocessing script, reducing the risk of training and serving skew.

6. API and Containerisation

The FastAPI service exposes /health, /predict, and /metrics. The prediction endpoint accepts the 13 feature fields, validates their types and ranges, loads the serialized pipeline, and returns a prediction, confidence, and model version.

The Dockerfile is based on python:3.12-slim. It installs pinned dependencies, downloads the dataset, trains the model during image build, and starts Uvicorn on port 8000. The example request in examples/predict.json can be posted to the running container.

The service emits request logs and uses Prometheus counters and a latency histogram. Missing model artifacts return HTTP 503 rather than a false success response; invalid input returns HTTP 422.

7. CI/CD and Automated Testing

The GitHub Actions workflow in .github/workflows/ci.yml checks out the repository, installs Python 3.12 dependencies, runs Ruff, downloads the dataset, runs Pytest, generates EDA plots, trains the models, and uploads the artifacts directory. Any failed command fails the workflow.

The tests cover preprocessing with missing values, schema validation for incomplete datasets, and the API health endpoint. The validated local test result is 3 passed. Additional tests can be added for prediction response shape and model-missing behavior.

The CI artifact upload provides a traceable record of generated model and EDA files for each successful workflow run.

8. Kubernetes Deployment and Monitoring

k8s/deployment.yaml creates a two-replica Deployment and a LoadBalancer Service. Readiness and liveness probes call /health. Resource requests and limits are defined to make scheduling behavior explicit.

For local deployment, Minikube can load the image, apply the manifest, wait for rollout completion, and expose the service with minikube service heart-disease-api --url. For cloud deployment, the image should be pushed to a registry and the image reference changed in the manifest.

k8s/prometheus.yaml provides a ServiceMonitor for clusters running Prometheus Operator. Grafana panels should display request volume, error count, and latency using prediction_requests_total and prediction_latency_seconds.

9. Limitations, Ethics, and Submission Evidence

The Cleveland dataset is small and historically collected. It may not represent current clinical practice or diverse populations. Model confidence is not calibrated clinical risk, and this API must not be used for diagnosis or treatment.

A production clinical system would require external validation, calibration, subgroup fairness analysis, access control, TLS, audit logging, drift monitoring, incident response, and human clinical oversight.

Submission evidence should include the generated EDA plots, MLflow comparison, passing CI run, Docker build and API response, Kubernetes rollout and service, and Prometheus/Grafana metrics. docs/EVIDENCE_CHECKLIST.md contains the exact capture list and a short video outline.

Local access: follow docs/SETUP.md. The API is available at http://localhost:8000 after starting Uvicorn. A public URL is not claimed because no cloud account or public cluster was provided.

Appendix A. EDA and Experiment Evidence

The following screenshots are generated from the project workspace and are included as direct submission evidence.

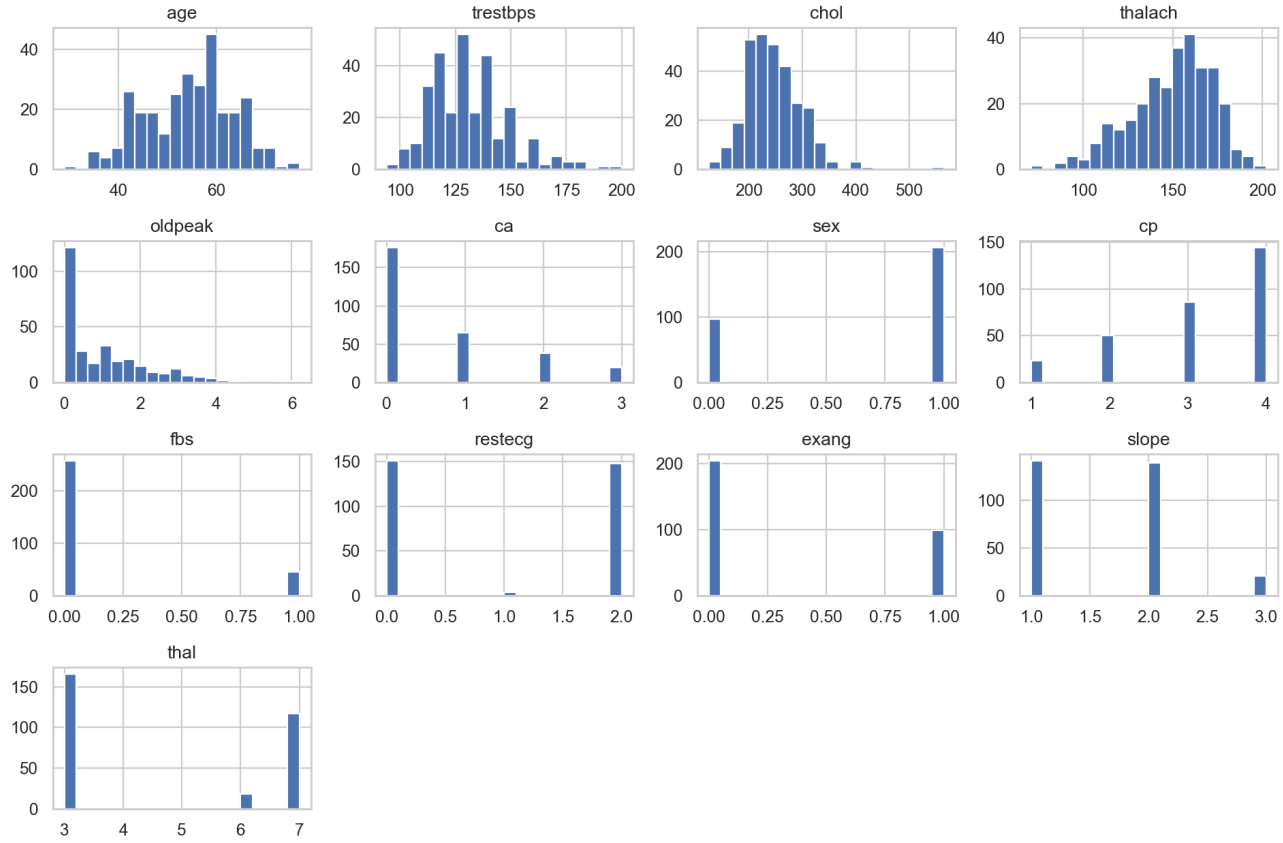


Figure A1. Feature histograms generated by the EDA script.

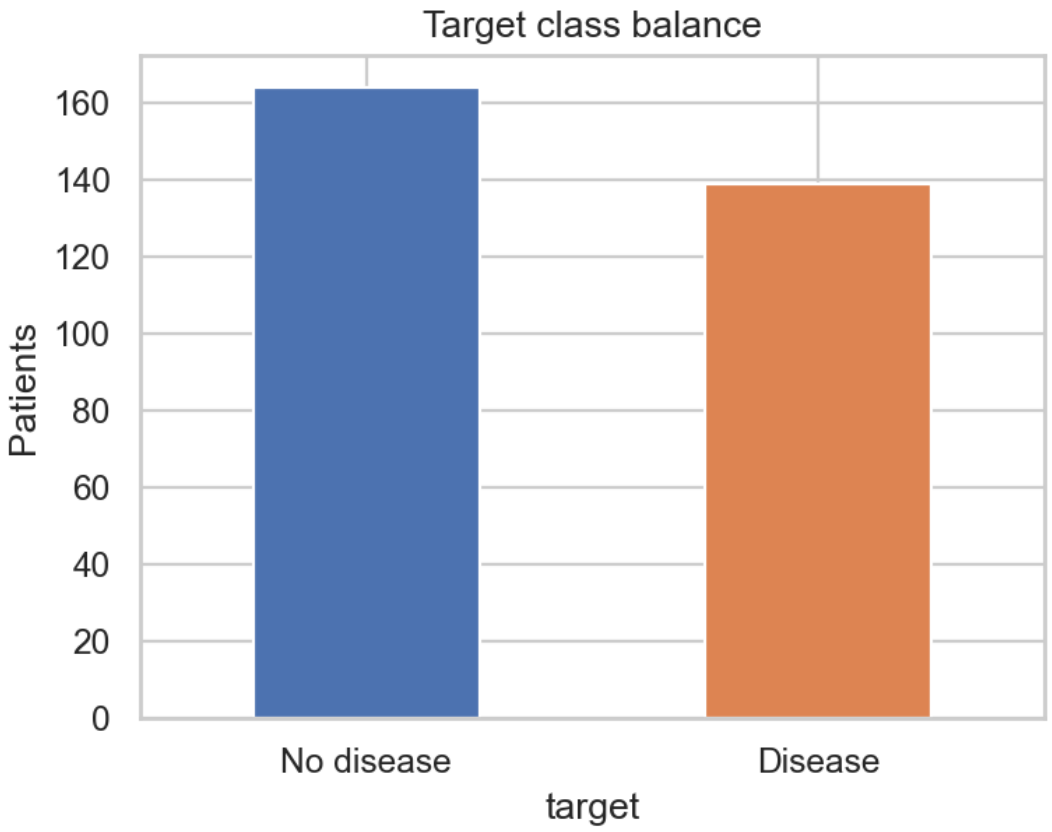


Figure A2. Target class balance for the Cleveland dataset.

Appendix B. Tracking and Continuous Integration Evidence

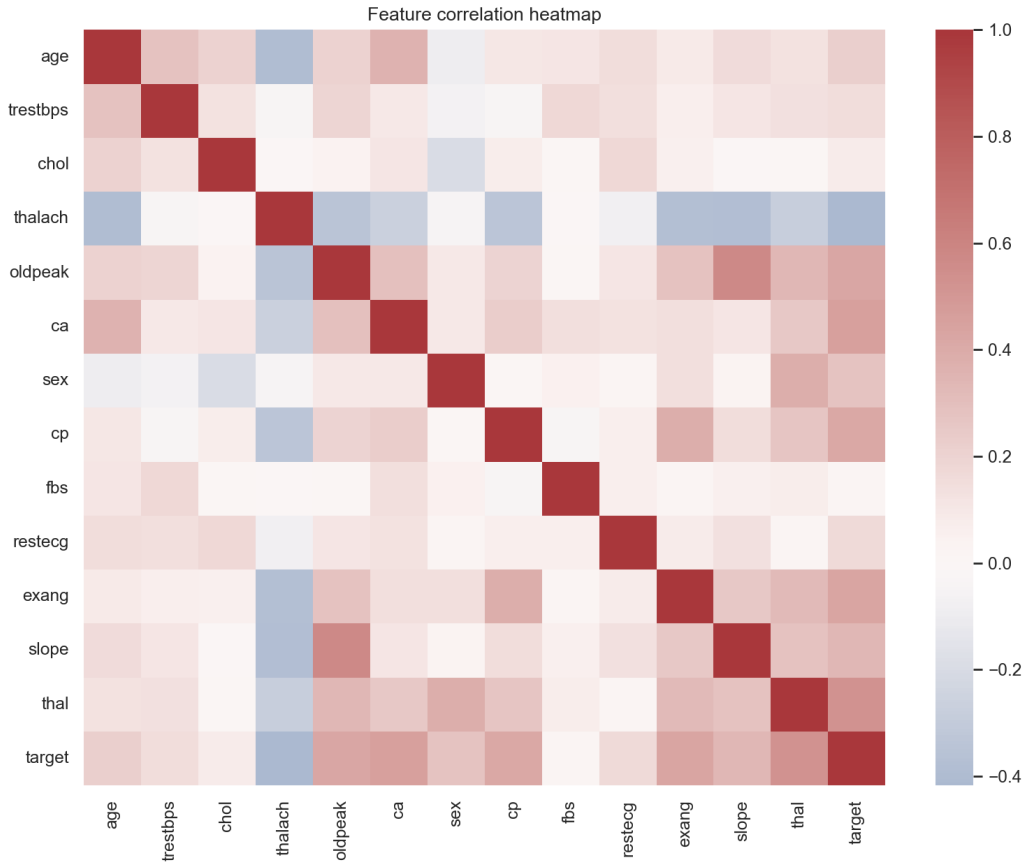


Figure B1. Correlation heatmap generated from the cleaned dataset.

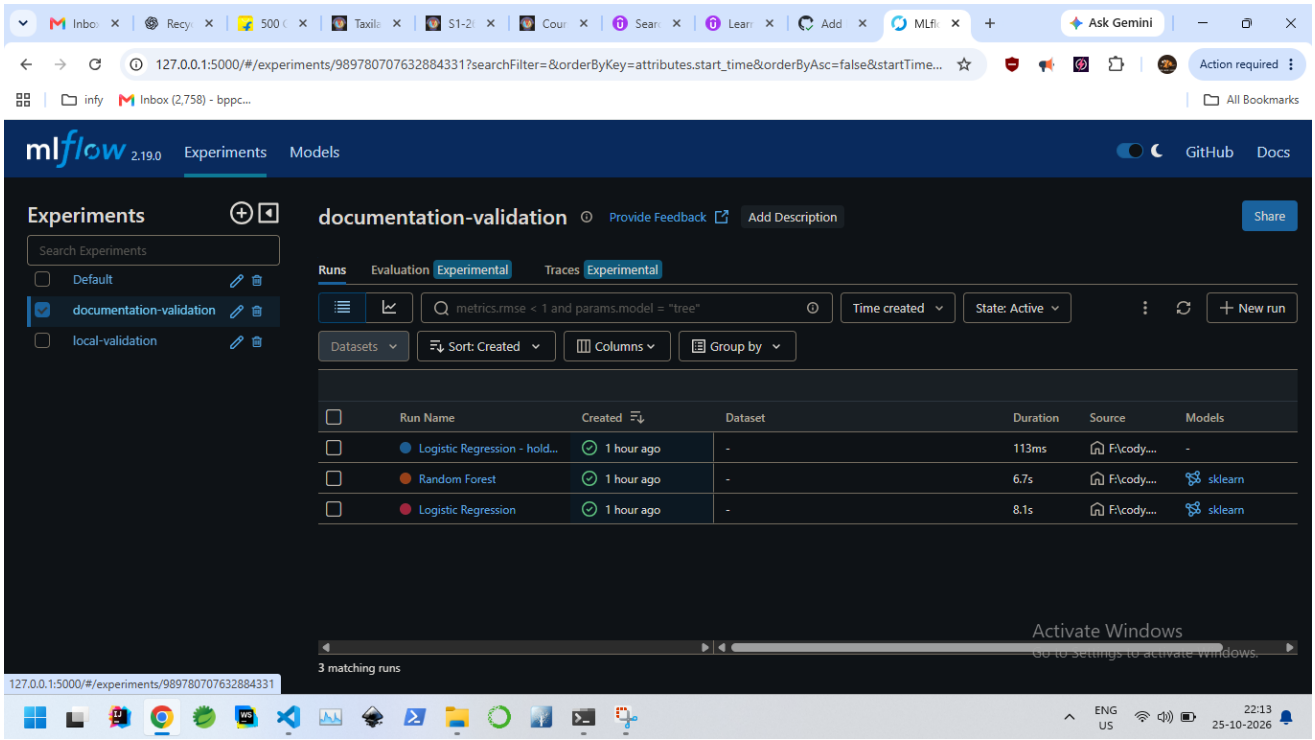


Figure B2. MLflow experiment tracking interface showing the experiment runs.

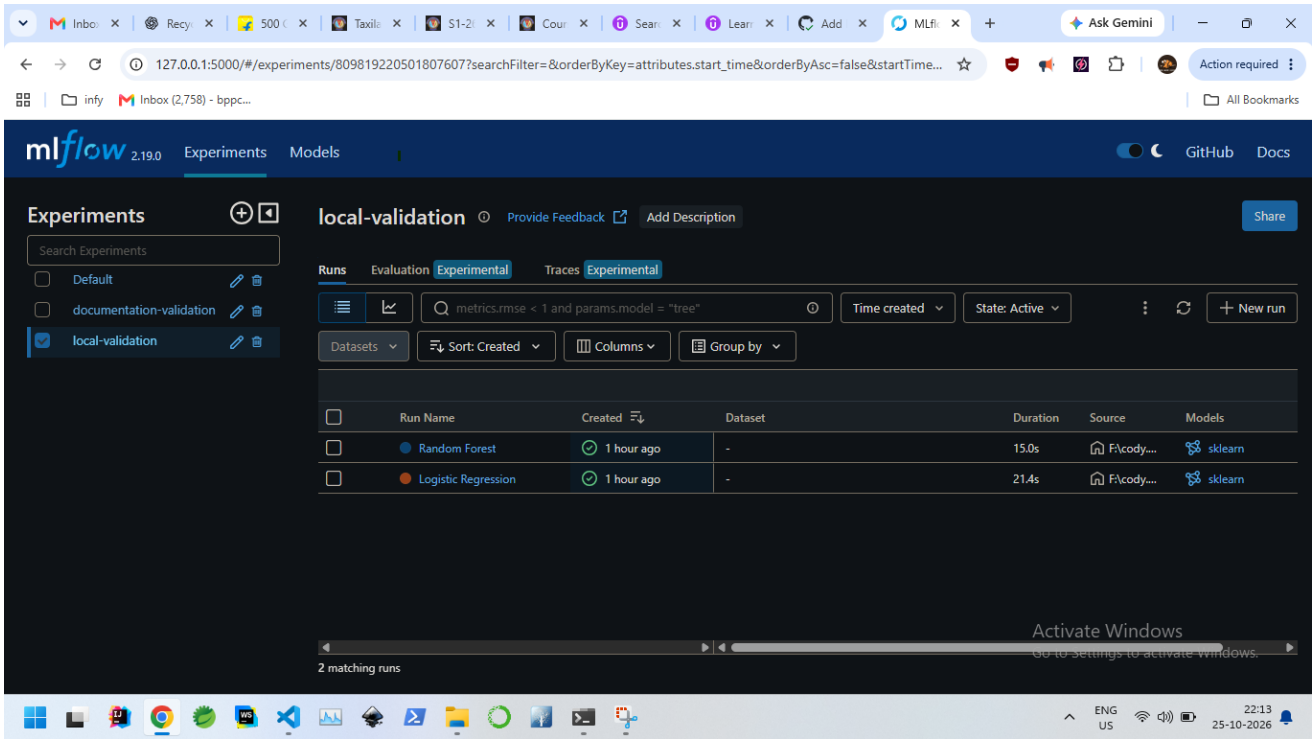


Figure B3. MLflow model run details and recorded metrics.

Appendix C. Containerised API Evidence

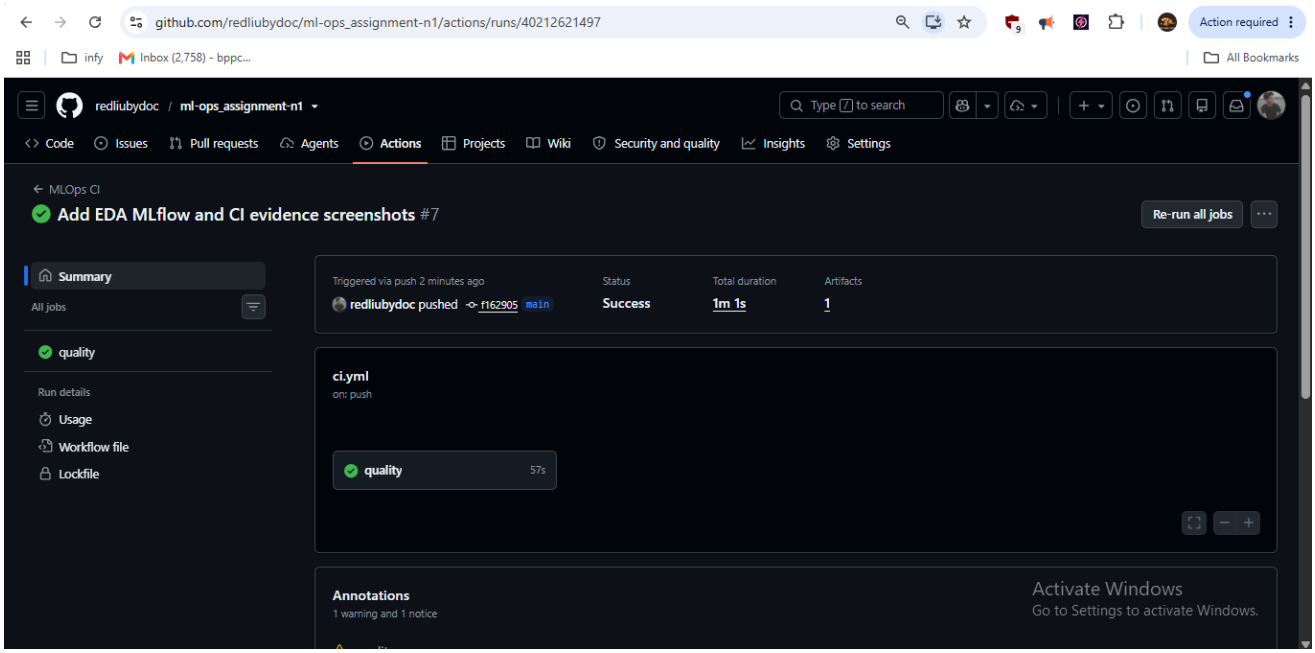


Figure C1. Successful GitHub Actions workflow.

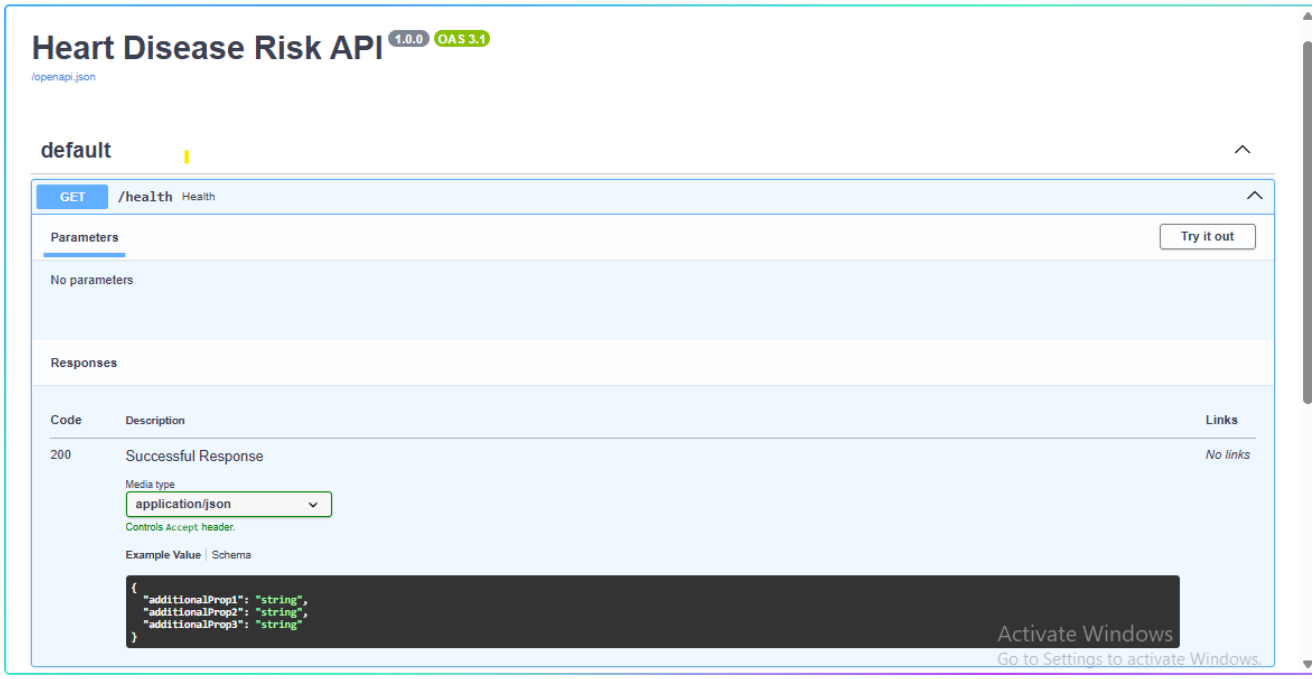


Figure C2. FastAPI Swagger UI served from the Docker container.